" AI-Driven Automated Video Generator for Content Creation "

An Application Development-II Report Submitted
In partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering -Artificial Intelligence and Machine Learning

by

Durishetty Anirudh	**-**	**22N31A6648**
Betham Vignesh Reddy	**-**	**22N31A6626**
B. Ranjith Kumar	**-**	**22N31A6622**

Under the Guidance of
Mrs. L. Melinda
Assistant Professor

DEPARTMENT OF CSE
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY
(Affiliated to JNTU, Hyderabad)

ACCREDITED by AICTE-NBA
Maisammaguda, Dhulapally post, Secunderabad-500014.

2024-2025

DECLARATION

I hereby declare that the project entitled **"PI-Driven Automated Video Generator for Content Creation"** submitted to **Malla Reddy College of Engineering and Technology,** affiliated t**o** Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering- Artificial Intelligence and Machine Learning** is a result of original research work done by me.

It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

Durishetty Anirudh (22N31A6648)

Betham Vignesh Reddy (22N35A6626)

B. Ranjith Kumar (22N31A6622)

MALLA REDDY COLLEGE
OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS INSTITUTION - UGC, GOVT. OF INDIA)
Affiliated to JNTUH; Approved by AICTE, NBA-Tier 1 & NAAC with A-GRADE | ISO 9001:2015

CERTIFICATE

This is to certify that this is the bonafide record of the project titled **"AI-Driven Automated Video Generator for Content Creation"** submitted by **Durishetty Anirudh** (22N31A6648), **Betham Vignesh Reddy** (22N31A6606), **B. Ranjith Kumar** (22N31A6612) of B.Tech in the partial fulfillment of the requirements for the degree of **Bachelor of Technology** in **Computer Science and Engineering - Artificial Intelligenceand Machine Learning**, Dept. of CI during the year 2024-2025. The results embodied in thisproject report have not been submitted to any other university or institute for the award of anydegree or diploma.

Dr. L. Melinda
Assistant Professor
INTERNAL GUIDE

Dr. D. Sujatha
Professor and Dean (CSE & ET)
HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

Date of Viva-Voce Examination held on: _____

ACKNOWLEDGEMENT

We feel honored and privileged to place our warm salutation to our college Malla Reddy College of Engineering and technology (UGC-Autonomous), our Director ***Dr. VSK Reddy*** who gave us the opportunity to have experience in engineering and profound technical knowledge.

We are indebted to our Principal ***Dr. S. Srinivasa Rao*** for providing us with facilities to do our project and his constant encouragement and moral support which motivated us to move forward with the project.

We would like to express our gratitude to our Head of the Department ***Dr. D. Sujatha,*** Professor and Dean (CSE&ET) for encouraging us in every aspect of our system development and helping us realize our full potential.

We would like to express our sincere gratitude and indebtedness to our project supervisor ***Dr. L. Melinda,*** Assistant Professor for her valuable suggestions and interest throughout the course of this project.

We convey our heartfelt thanks to our Project Coordinator***, Mr. T Hari Babu,*** Assistant Professor for allowing for his regular guidance and constant encouragement during our dissertation work

We would also like to thank all supporting staff of department of CI and all other departments who have been helpful directly or indirectly in making our Application Development-II a success.

We would like to thank our parents and friends who have helped us with their valuable suggestions and support has been very helpful in various phases of the completion of the Application Development-II.

By:

Durishetty Anirudh (22N31A6644)

Betham Vignesh Reddy (22N35A6606)

B. Ranjith Kumar (22N31A6612)

ABSTRACT

This project presents an innovative solution to the challenges of content creation in the rapidly evolving social media landscape. Traditional methods of video production are often manual, time-consuming, and require multiple platforms for scriptwriting, voiceovers, captioning, and video editing. To address these issues, we have developed a fully automated video generation system that streamlines the process from keyword-based script generation to final video rendering. The system integrates several advanced technologies, including ChatGPT for automatic script generation, EdgeTTS for realistic voiceovers, and Whisper for generating captions. Python-based video processing tools are used to combine these elements into a polished final video. The entire process is made accessible through a user-friendly Streamlit interface, which allows users to customize their video outputs with minimal effort. This automated approach enhances efficiency and offers significant time savings for content creators, while maintaining flexibility for customization based on individual needs. The system's seamless integration of cutting-edge technologies provides a comprehensive solution for effortless content creation in today's fast-paced digital environment.

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

Small content creators face a significant challenge in transforming their ideas into engaging video content. This time-consuming and labor-intensive process can be overwhelming for those without the resources or expertise of larger content producers. As a result, these creators are left with limited time and energy to focus on developing their creative vision. The need for a more efficient and streamlined approach to video creation is clear.

1.2 Objectives

Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the **AI-Driven Automated Video Generator for Content Creation**. The system aims to automate the entire video production process by leveraging artificial intelligence to generate scripts, synthesize voiceovers, create captions, and compile videos seamlessly. The document serves as a comprehensive guide detailing the system’s objectives, functionalities, constraints, and implementation specifics, ensuring clarity for all stakeholders involved in the project.

1.3 Summary

This chapter introduces the AI-Driven Automated Video Generator, a system that aims to automate the entire video production process. The problem statement highlights the shortcomings of existing multimedia production methods and the need for a more efficient, scalable, and accessible approach. The objectives outline the functional and non-functional requirements for the system, which include generating scripts, synthesizing voiceovers, creating captions, and compiling videos seamlessly. The chapter establishes the motivation behind creating this tool and sets the stage for the project's scope and goals. The summary concludes by previewing the next chapter, which will explore existing literature and systems that influenced the proposed solution. Overall, Chapter 1 provides a comprehensive overview of the AI-Driven Automated Video Generator's objectives and design principles.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing System:

Currently, video content creation is heavily dependent on manual intervention, with most creators using a combination of tools for scriptwriting, voiceover recording, video editing, and caption synchronization. Some existing systems include:

- **Traditional Methods:**
  - Manual scriptwriting and research.
  - Hiring voice artists for narration.
  - Using software like Adobe Premiere Pro or Final Cut Pro for video editing.
  - Manually adding captions using video editing tools.
- **AI-Assisted Solutions:**
  - Some platforms offer text-to-speech conversions but lack video integration.
  - Certain AI-driven tools generate captions but do not sync them with videos effectively.
  - Existing solutions do not provide full automation from script generation to video rendering with minimalistic approach.

2.1.1 Drawbacks of existing system:

The existing approaches to video production come with several challenges:

- **Time-Consuming Workflow:** Creating a single professional-quality video can take hours or even days due to manual effort at multiple stages.
- **Fragmented Toolsets:** Users rely on multiple applications for scriptwriting, voiceovers, video editing, and captioning, leading to inefficiencies.
- **High Costs:** Professional content creation tools and hiring voice artists add significant expenses to the production process.
- **Inconsistent Quality:** Manually created content often lacks uniformity, affecting branding and engagement consistency.

2.2 Proposed System:

- To address these limitations, the AI-Driven Automated Video Generator offers a **fully integrated and automated solution**:
- **AI-Powered Scriptwriting:** Generates topic-based scripts using **ChatGPT**, ensuring engaging and well-structured content.
- **Realistic Voiceover Synthesis:** Uses **EdgeTTS** to produce human-like narration with multiple voice and language options.
- **Accurate Captioning:** Implements **Whisper** to create perfectly synchronized captions for accessibility and engagement.
- **Automated Video Retrieval:** Searches and selects suitable background footage from **Pexels API** based on AI-analyzed keywords.
- **One-Click Video Rendering:** Combines all elements using **MoviePy and FFmpeg**, producing high-quality MP4 videos ready for sharing.
- **Customizability:** Allows users to edit generated scripts, select different voiceovers, adjust captions, and preview the video before finalization.
- By combining AI-based automation with a user-friendly interface, the system significantly reduces video production time while ensuring high-quality output at minimal cost.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 Introduction:

"AI-Driven Video Generation Platform," an innovative content creation system designed to address the shortcomings of existing multimedia production methods. By leveraging the power of Generative AI, this system aims to provide a more efficient, scalable, and accessible approach to transforming text into high-quality video content. This section will outline the core features, functionalities, and design principles of the proposed system, providing a comprehensive overview of how this AI-driven platform will achieve its objectives.

3.2 Software and Hardware Requirements

Software Requirements:

- **Operating System:** Compatible with Windows 10/11, Linux, and macOS.
- **Programming Language:** Python 3.11.
- **Web Framework:** Streamlit for UI and API interactions.
- **Libraries and Dependencies:** OpenAI API, EdgeTTS, Whisper, MoviePy, FFmpeg, NumPy, Pandas.
- **Cloud and Storage:** AWS or Google Drive for storing generated videos.
- **Version Control:** GitHub for source code management and collaboration.
- **Database:** Log storage and retrieval mechanisms for AI-generated scripts, captions, and metadata.

Hardware Requirements:

- **Processor:** Ryzen 7 5800H.
- **RAM:** Minimum 16GB (Recommended 32GB for high-performance tasks).
- **Storage:** 20GB free space (SSD preferred for faster data access).
- **GPU:** NVIDIA RTX 3050 4GB or equivalent for AI model acceleration.
- **Internet Connection:** High-speed internet for real-time API calls and media processing.

3.3 Functional and Non-Functional Requirements

- **Functional:**
    - Generate scripts
    - Synthesize voiceovers
    - Create captions
    - Compile videos seamlessly
- **Non-Functional:**
    - Efficient (implying a requirement for speed or performance)
    - Scalable (implying a requirement for flexibility and adaptability)
    - Accessible (implying a requirement for ease of use)

3.4 Other Requirements

- User must have Ollama installed and the model tinyllama preloaded.
- App must be run with appropriate permissions to spawn subprocesses.

3.5 Summary

PromptCraft provides a complete local solution with minimal dependencies and a seamless interface for prompt generation.

CHAPTER-4

SYSTEM DESIGN

4.1 System Architecture

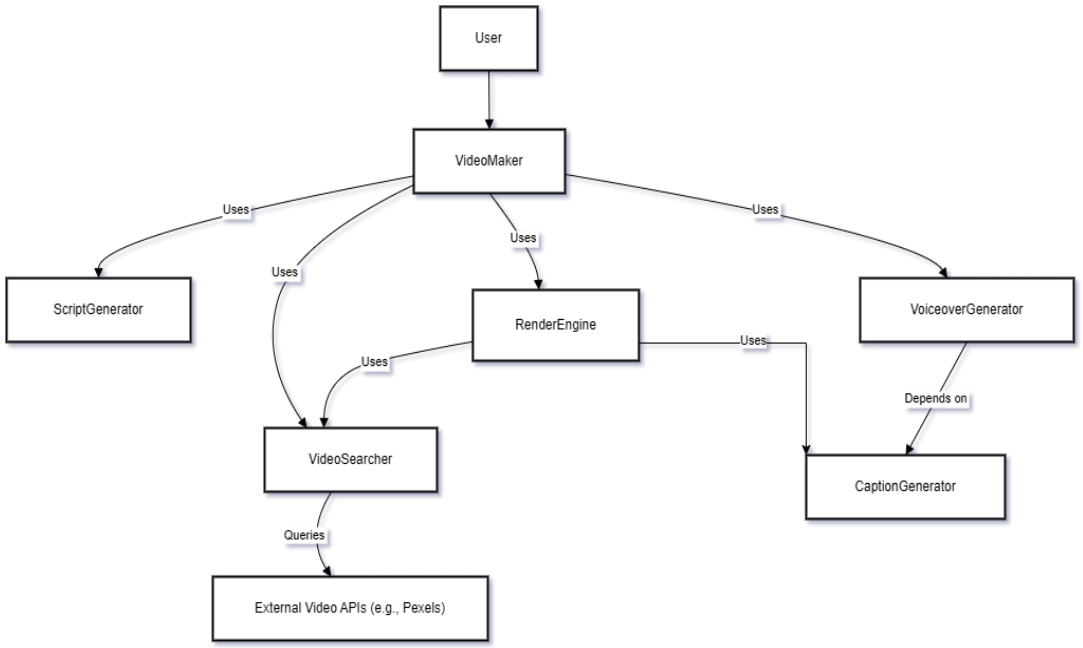


Fig 4.1 System Architecture of AI-Driven Automated Video Generator for Content Creation

4.2 Data flow diagram

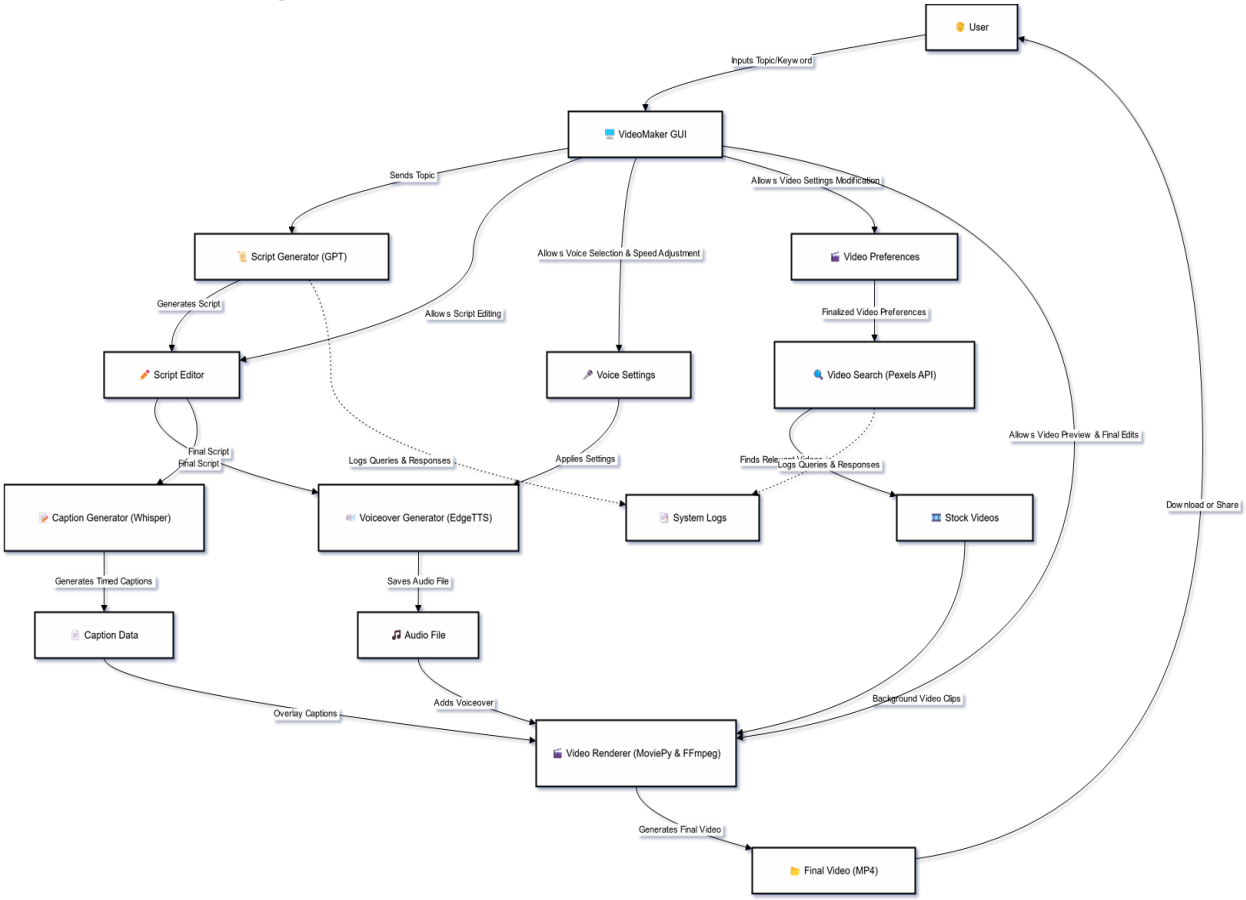


Fig 4.2 Data Flow Diagram for AI-Driven Automated Video Generator for Content Creation

4.3 UML Diagrams

The unified modeling is a standard language for specifying, visualizing, constructing and documenting the system and its components is a graphical language which provides a vocabulary and set of semantics and rules. The UML focuses on the conceptual and physical representation of the system. It captures the decisions and understandings about systems that must be constructed. It is used to understand, design, configure and control information about the systems.

4.3.1 Use case diagram

A use case diagram is a graph of actors set of use cases enclosed by a system boundary, communication associations between the actors and users and generalization among use cases. The use case model defines the outside (actors) and inside (use case) of the system's behavior.

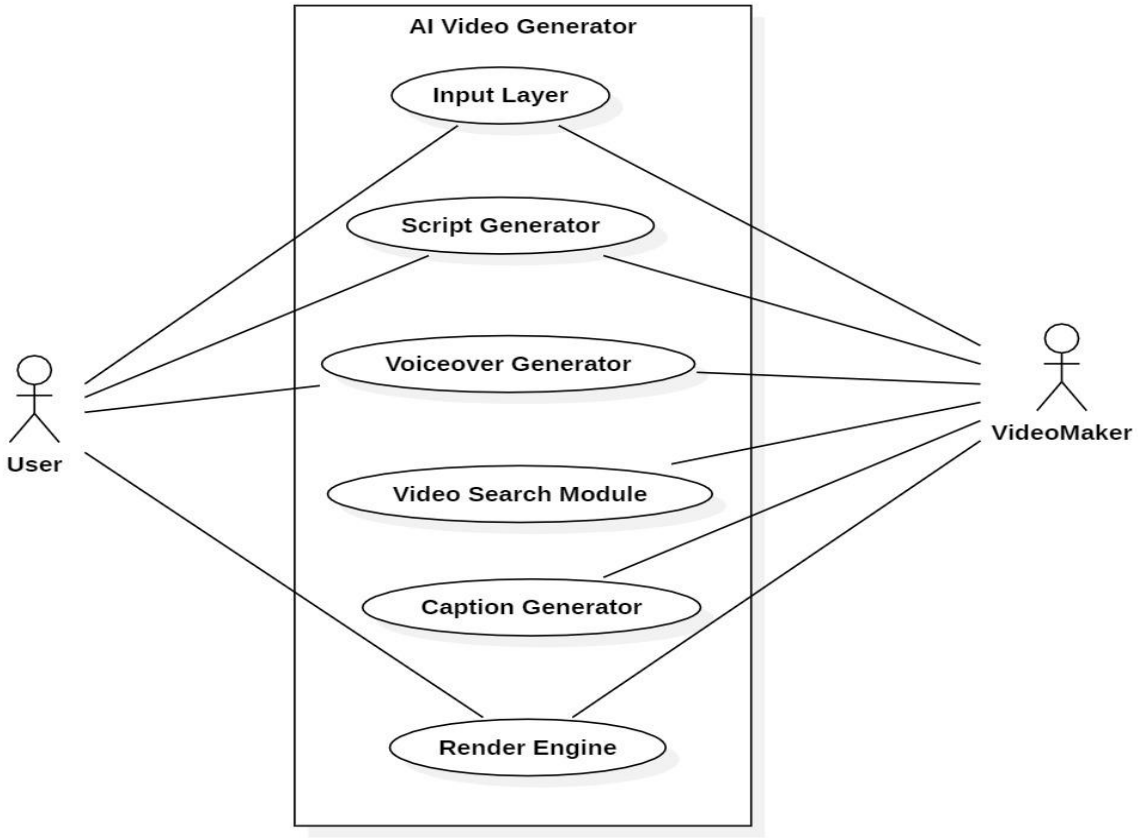


Fig 4.3.1 Use Case Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.2 Class Diagram

A class is a representation of an object and, in many ways; it is simply a template from which objects are created. Classes form the main building blocks of an object-oriented application. Although thousands of students attend the university, you would only model one class, called Student, which would represent the represent the entire collection of students.

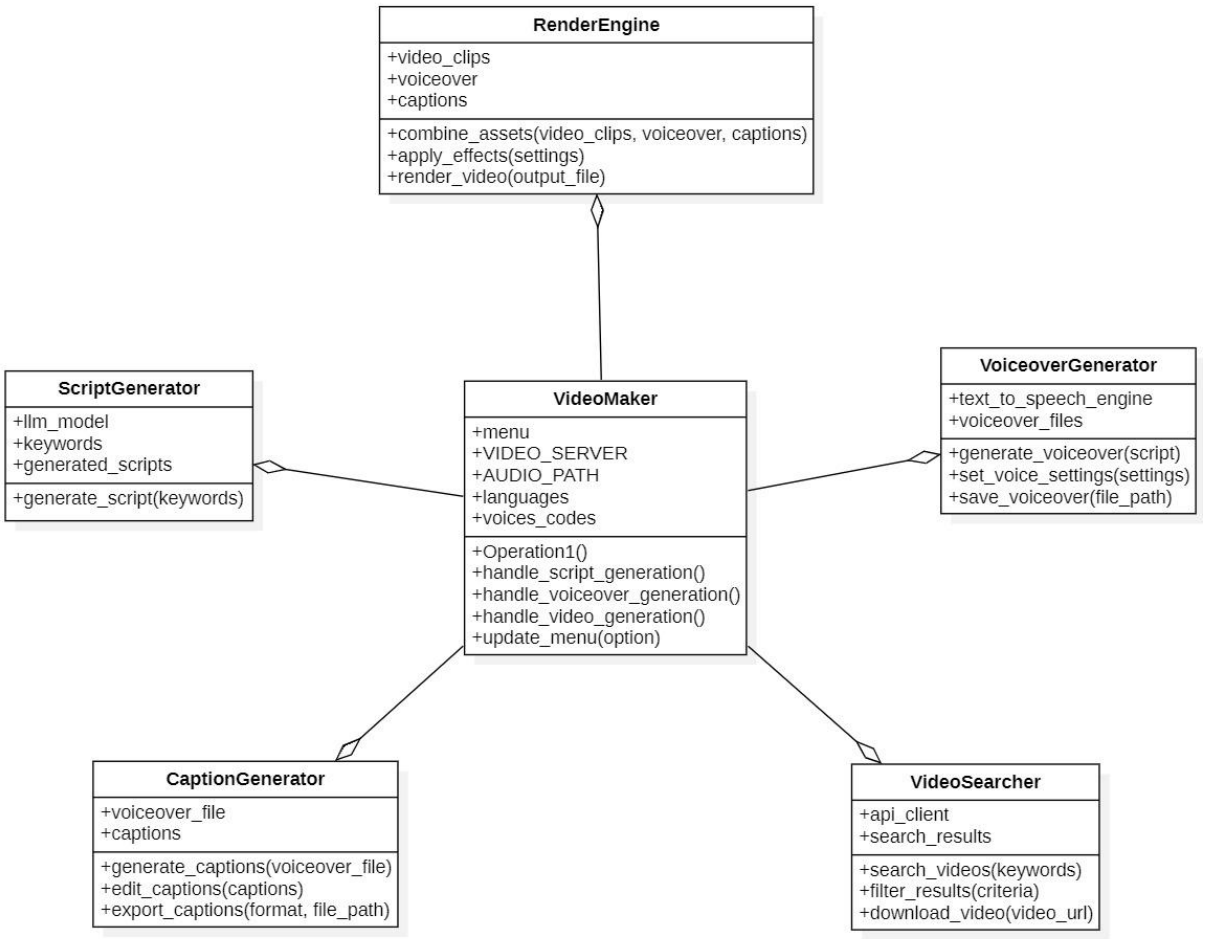


Fig 4.3.2 Class Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.3 Sequence Diagram

Sequence diagram is used to represent the flow of messages, events and actions between the objects or components of a system. Time is represented in the vertical direction showing the sequence of interactions of the header elements, which are displayed horizontally at the top of the diagram.

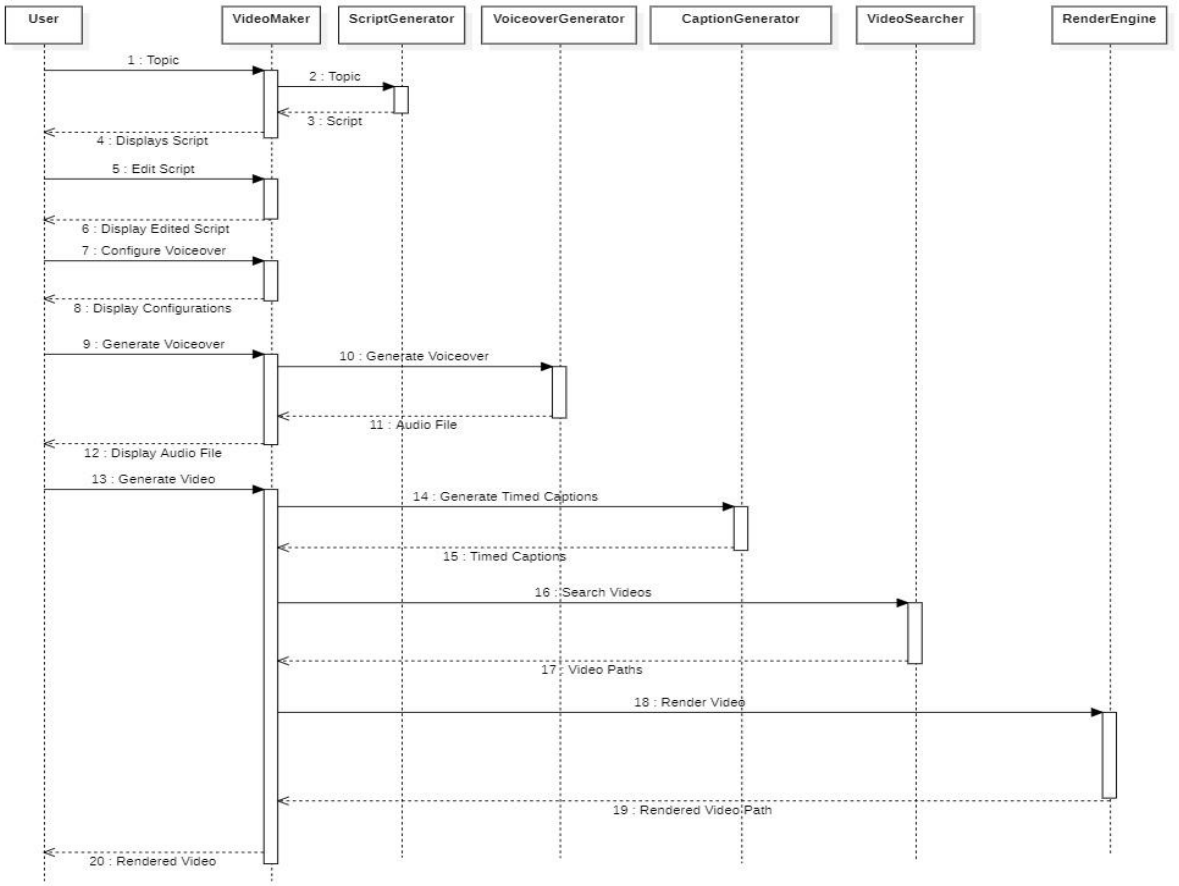
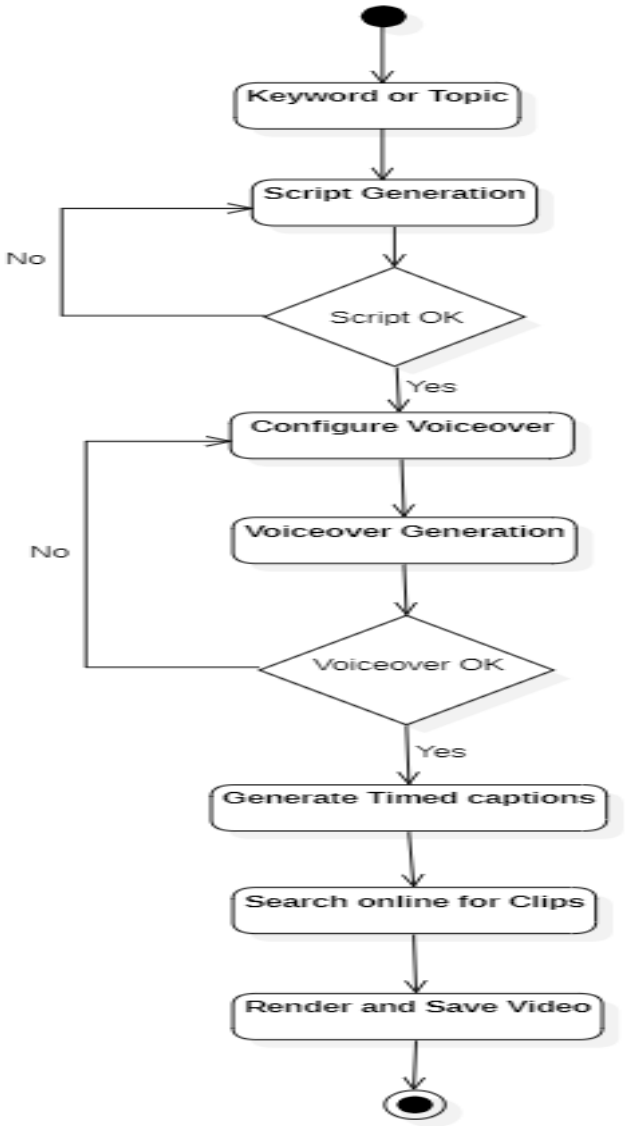


Fig 4.3.3 Activity Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.4 Activity Diagram

Activity diagram represents the business and operational workflows of a system. An Activity diagram is a dynamic diagram that shows the activity and the event that causes the object



to be in the particular state.

Fig 4.3.4 Activity Diagram for AI-Driven Automated Video Generator for Content Creation

4.4 Summary

A clean separation between UI logic, API handling, and model processing ensures easy extensibility and maintainability.

CHAPTER-5
IMPLEMENTATION

5.1 Algorithms

The AI-Driven Automated Video Generator follows a step-by-step process powered by intelligent algorithms:

1. **Script Generation**: The system uses a language model (ChatGPT) to create a meaningful script from a user-provided topic.
2. **Voiceover Creation**: The script is converted into speech using EdgeTTS, with options for different voices and accents.
3. **Caption Generation**: Whisper processes the audio to create accurate, time-synced captions.
4. **Video Clip Search**: The script and captions are analyzed to generate search terms, which are used to find relevant video clips from the Pexels API.
5. **Clip Matching and Merging**: Retrieved video clips are matched with caption timings. If gaps exist, a simple merging technique is used to fill them.
6. **Final Video Rendering**: All components — audio, video, and captions — are combined using MoviePy and FFmpeg to produce the final MP4 video.

5.1.1 Code

```
import streamlit as st
import Passwords  # Assuming you need this elsewhere
import os
from utility.captions.timed_captions_generator import generate_timed_captions
from utility.render.render_engine import get_output_media
from utility.video.background_video_generator import generate_video_url
from utility.video.video_search_query_generator import getVideoSearchQueriesTimed,
merge_empty_intervals  # Assuming you need this elsewhere
st.set_page_config(layout="wide")
st.sidebar.subheader = "Stage"
if "menu" not in st.session_state:
    st.session_state["menu"] = "Script Generation"
menu = st.sidebar.selectbox(
    "",
```

```
    ["Script Generation", "Voiceover", "Video Generation"],
    index=["Script Generation", "Voiceover", "Video Generation"].index(st.session_state["menu"])
)
VIDEO_SERVER = "pexel"
AUDIO_PATH = "audio_tts.wav"
languages = ["English (United States)",
             "English (United Kingdom)",
             "English (Australia)",
             "English (Canada)",
             "English (India)",
             "English (New Zealand)",
             "English (Ireland)",
             "English (South Africa)"
             ]
voices_codes = {
    "English (United States)": [
        ["en-US-GuyNeural", "en-US-ChristopherNeural"],  # Male voices
        ["en-US-AriaNeural", "en-US-JennyNeural", "en-US-AmberNeural", "en-US-AnaNeural"]  # Female voices
    ],
    "English (United Kingdom)": [
        ["en-GB-RyanNeural", "en-GB-GeorgeNeural"],  # Male voices
        ["en-GB-LibbyNeural", "en-GB-SoniaNeural"]  # Female voices
    ],
    "English (Australia)": [
        ["en-AU-WilliamNeural"],  # Male voices
        ["en-AU-NatashaNeural"]  # Female voices
    ],
    "English (Canada)": [
        ["en-CA-LiamNeural"],  # Male voices
        ["en-CA-ClaraNeural"]  # Female voices
```

```
    ],
    "English (India)": [
        ["en-IN-PrabhatNeural"],  # Male voices
        ["en-IN-NeerjaNeural"]  # Female voices
    ],
    "English (New Zealand)": [
        ["en-NZ-MitchellNeural"],  # Male voices
        ["en-NZ-MollyNeural"]  # Female voices
    ],
    "English (Ireland)": [
        ["en-IE-ConnorNeural"],  # Male voices
        ["en-IE-EmilyNeural"]  # Female voices
    ],
    "English (South Africa)": [
        ["en-ZA-ThembaNeural"],  # Male voices
        ["en-ZA-TessaNeural"]  # Female voices
    ]
}
if menu == "Script Generation":
    st.title("Script Generation and Voiceover")
    st.markdown(
        """
        <style>
        .custom-label {
            font-size: 20px;
            font-weight: bold;
        }
        </style>
        """,
        unsafe_allow_html=True,)
    # Initialize session state variables
```

```
    if "textarea_value" not in st.session_state:
        st.session_state["textarea_value"] = ""  # Default value for script
    if "audio" not in st.session_state:
        st.session_state["audio"] = 0 # Default value for
    # Function to generate the script
    def generate_script(user_prompt):
        from utility.script.script_generator import generate_script
        return generate_script(user_prompt)
    # Create columns for input and button
    col1, col2 = st.columns([4, 1])  # Adjust column widths as needed
    # Capture user prompt
    with col1:
        user_prompt = st.text_input(
            "Topic",
            placeholder="Enter your topic here"
        )
    # Add button to generate script
    with col2:
        st.markdown("<br>", unsafe_allow_html=True)  # Add space to align with text input
        if st.button("Generate Script"):
            if user_prompt.strip():  # Ensure the input is not empty
                st.session_state["textarea_value"] = generate_script(user_prompt)
            else:
                st.warning("Please enter a valid topic to generate a script.")
    # Render the updated text area value
    script = st.text_area("Script", height=300, value=st.session_state["textarea_value"])
    if st.session_state["textarea_value"] != "":
        def script_next():
            st.session_state["menu"] = "Voiceover"
        st.button("Next",on_click=script_next)
if menu == "Voiceover" :
```

```
if st.session_state["textarea_value"] != "":
    # script = "Get ready for some incredible turtle facts! 1. Turtles have been around for 220 million years, surviving alongside dinosaurs. 2. They can't retract their limbs into their shells for protection like other turtles might. 3. Sea turtles can travel up to 22 miles per hour, faster than some cars! 4. Leatherback sea turtles are the largest turtle species, sometimes growing over 6 feet long. 5. A turtle's shell is not just one piece; it's made up of over 50 bones. 6. Despite their slow speed, turtles can live for over 100 years, making them some of the longest-living creatures. 7. Turtle eggs can take up to 100 days to hatch! 8. Turtles play a key role in the ecosystem by controlling jellyfish and sea grass populations. Fascinating, huh?"
    st.title("Voiceover Generation")
    dash, configuration = st.columns([70,30])
    dash.markdown(
        """
        <h3 style="margin-bottom: 10px;">Your script</h3>
        """,
        unsafe_allow_html=True,
    )
    # Display the script
    dash.markdown(
        f"""
        <div style="margin-top: 0px; font-size: 16px; line-height: 1.5; text-align:justify;">
            {st.session_state["textarea_value"]}
        </div>
        """,
        unsafe_allow_html=True,
    )
    col1, col2= configuration.columns([1,3])
    col2.subheader("Audio Settings")
    lang = col2.selectbox("Select Language", languages)
    col2.markdown("<br>", unsafe_allow_html=True)
    gender = col2.selectbox("Select Voice Gender",["Male", "Female"])
```

```
        col2.markdown("<br>", unsafe_allow_html=True)
         if gender == "Male":
            voices = voices_codes[lang][0]
        else:
            voices = voices_codes[lang][1]
        voice = col2.selectbox("Select Voice Gender", voices)
        col2.markdown("<br>", unsafe_allow_html=True)
        if col2.button("Generate Voiceover"):
                import asyncio
                from utility.audio.audio_generator import generate_audio
                asyncio.run(generate_audio(st.session_state["textarea_value"], AUDIO_PATH, voice))
                # asyncio.run(generate_audio(script, AUDIO_PATH, voice))
                st.session_state["audio"] = 1
        dash.markdown("<br>", unsafe_allow_html=True)
        if os.path.exists(AUDIO_PATH) and st.session_state["audio"] == 1:
            dash.subheader("Voicover Audio")
            dash.audio(AUDIO_PATH)
        col1,col2,col3,col4,col5,col6,col7 = dash.columns(7)
        def voice_prev():
            st.session_state["menu"] = "Script Generation"
        col1.button("Previous", on_click= voice_prev)
        if os.path.exists(AUDIO_PATH) and st.session_state["audio"] == 1:
            def voice_next():
                st.session_state["menu"] = "Video Generation"
            col2.button("Next",on_click=voice_next)
    else:
        st.title("Voiceover Generation")
        st.warning("Please complete the Script Generation and you will be able to generate audio here.
")
if menu == "Video Generation":
    st.title("Video Generation")
```

```
    if st.session_state["audio"] == 1 and st.session_state["textarea_value"] != "":
        st.markdown("All Set! Ready for the magic?")
        def generate_video():
            timed_captions = generate_timed_captions(AUDIO_PATH)
            print(timed_captions)
            search_terms = getVideoSearchQueriesTimed(st.session_state["textarea_value"],
timed_captions)
            print(search_terms)
            background_video_urls = None
            if search_terms is not None:
                background_video_urls = generate_video_url(search_terms, VIDEO_SERVER)
                print(background_video_urls)
            else:
                print("No background video")
            background_video_urls = merge_empty_intervals(background_video_urls)
            if background_video_urls is not None:
                video = get_output_media(AUDIO_PATH, timed_captions, background_video_urls,
VIDEO_SERVER)
                print(video)
        st.button("Next",on_click=generate_video)
        if os.path.exists("rendered_video.mp4"):
            st.video("rendered_video.mp4")
    else:
        st.warning("Please complete the Script Generation and Voiceover Generation to access Video
Generation")
```

5.2 Architectural Components

- **Frontend (UI Layer)**: Built with Streamlit for stage-wise interaction — Script, Voiceover, and Video Generation.
- **Controller Layer**: Manages session state, workflow logic, and integrates AI services.
- **Script Generator**: Uses ChatGPT via OpenAI API to create structured scripts.
- **Voiceover Generator**: Converts script to speech using EdgeTTS with multiple voice options.
- **Caption Generator**: Uses Whisper to generate time-aligned subtitles from audio.
- **Video Asset Manager**: Retrieves stock footage using Pexels API and aligns it with captions.
- **Rendering Engine**: Combines audio, captions, and video using MoviePy and FFmpeg.
- **Configuration & Storage**: Handles audio/video paths, user preferences, and API constants.

5.3 Feature Extraction

- The system performs implicit feature extraction at multiple stages to enable intelligent video generation:
- Text Feature Extraction: Semantic keywords and context are extracted from user-generated scripts to generate search queries for relevant video content.
- Audio Feature Extraction: The Whisper model analyzes the audio waveform to extract timing and phonetic features, enabling accurate generation of time-aligned captions.
- Temporal Feature Extraction: Caption timestamps and voiceover pacing are used to segment the video timeline, allowing precise alignment of video clips with the narration.
- Visual Feature Matching (via Pexels API): Extracted keywords and scene descriptions are used to retrieve stock footage that visually matches the script context.
- These features are critical for automating content alignment, improving relevance, and ensuring a seamless viewing experience.

5.4 Packages / Libraries Used

- Streamlit is used to create a clean and interactive user interface for each stage of the app. OpenAI (ChatGPT API) is used to generate structured scripts from user-inputted topics using natural language processing.
- EdgeTTS is used for converting text scripts into natural-sounding voiceovers with customizable voices.
- Whisper is implemented to generate accurate, timed subtitles from the generated audio using speech recognition.
- MoviePy is used for video editing tasks such as merging video clips, adding audio, and overlaying captions.
- FFmpeg works as a backend tool for MoviePy to process, convert, and render high-quality video outputs.
- NumPy helps manage numerical data during the captioning and rendering stages.
- Pandas is used for organizing and handling structured caption and script data.
- GitHub is used for version control, collaboration, and codebase management.
- Pexels API is used to retrieve relevant royalty-free video clips based on search terms generated from script and captions.

5.5 Output Screens

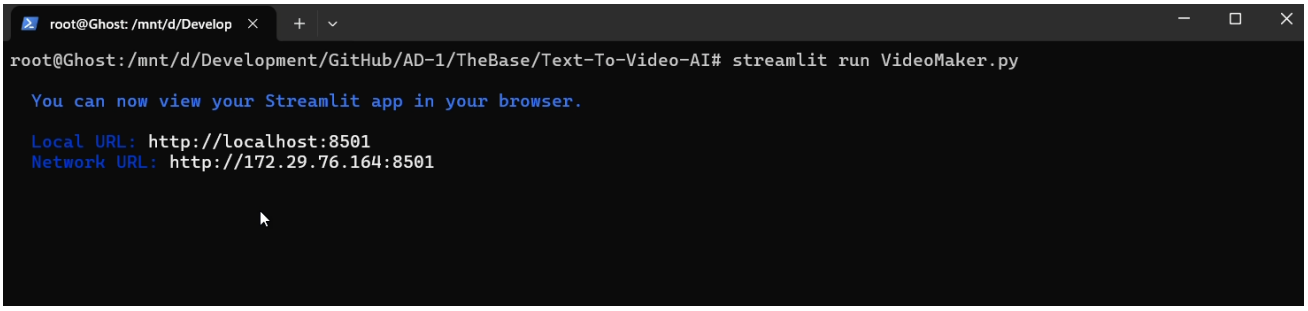


Fig 5.5(a) Initialization of application

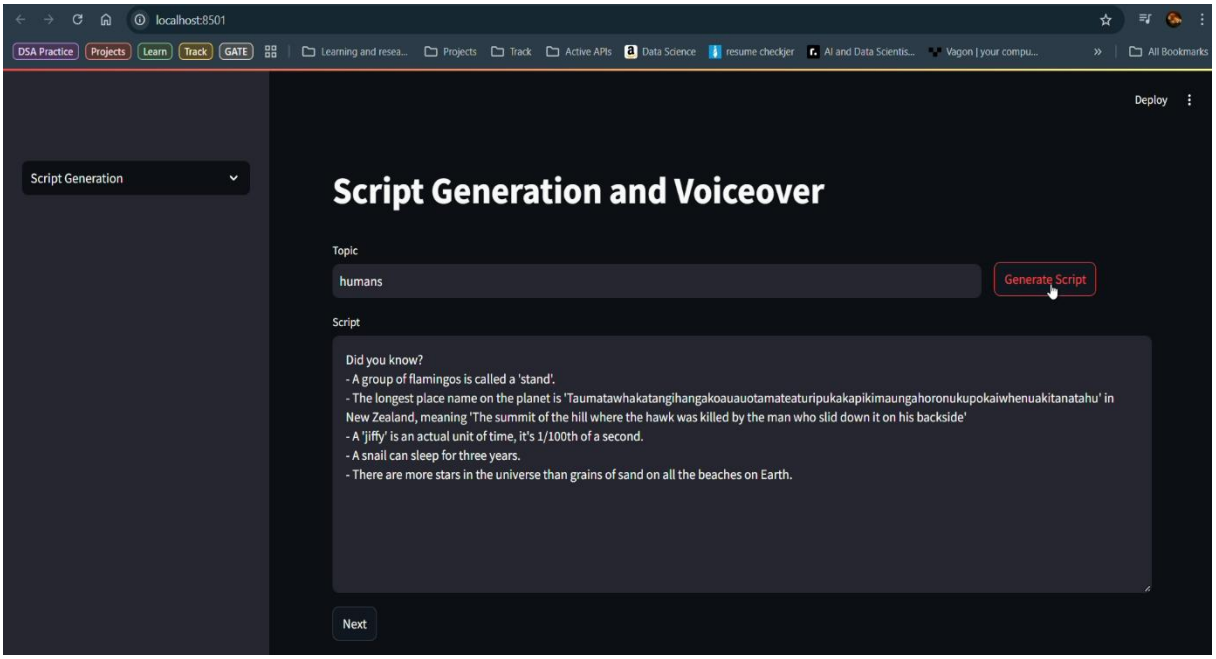


Fig 5.5(b) Script Generation phase

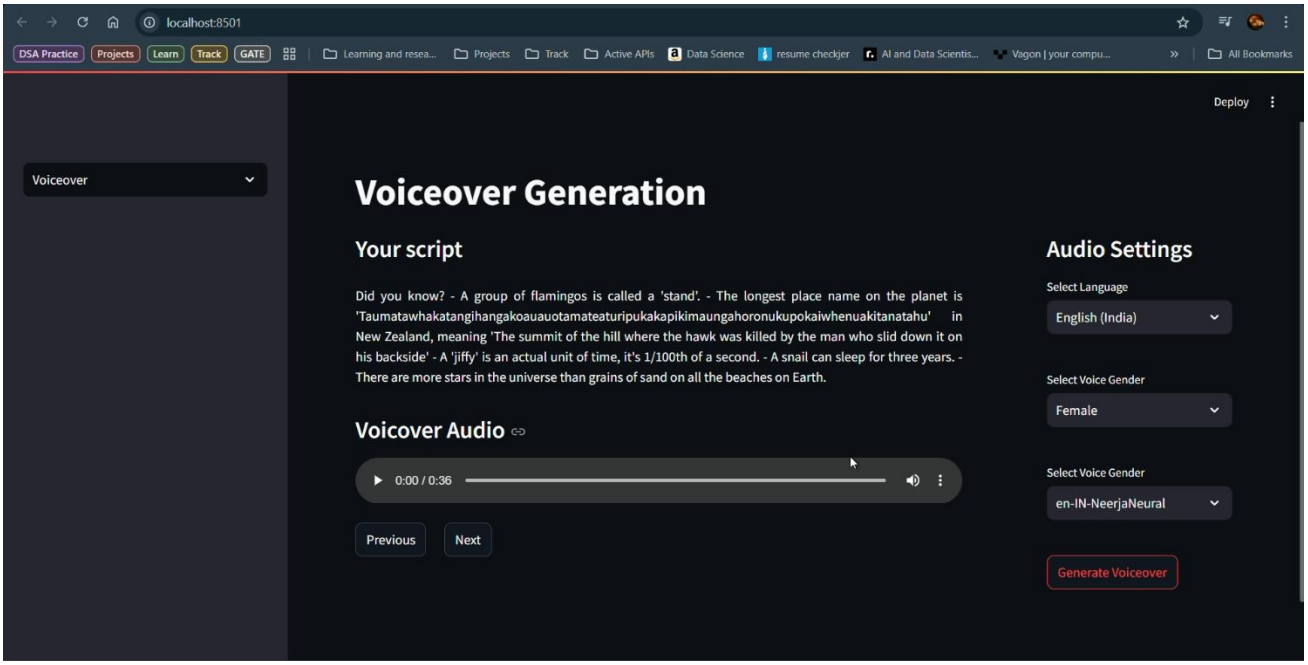


Fig 5.5(c) Voiceover Genration Phase

Fig 5.5(d) Video Rendering Phase Shell Interface

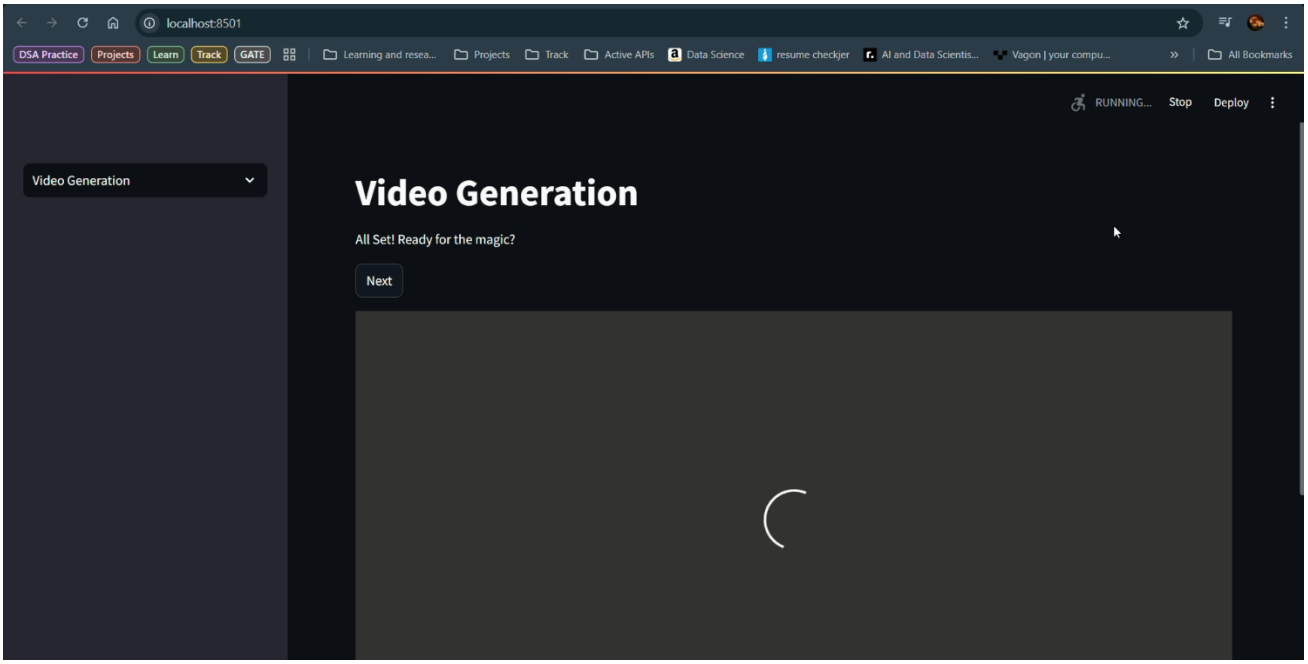


Fig 5.5(e) Video Generation Phase on GUI

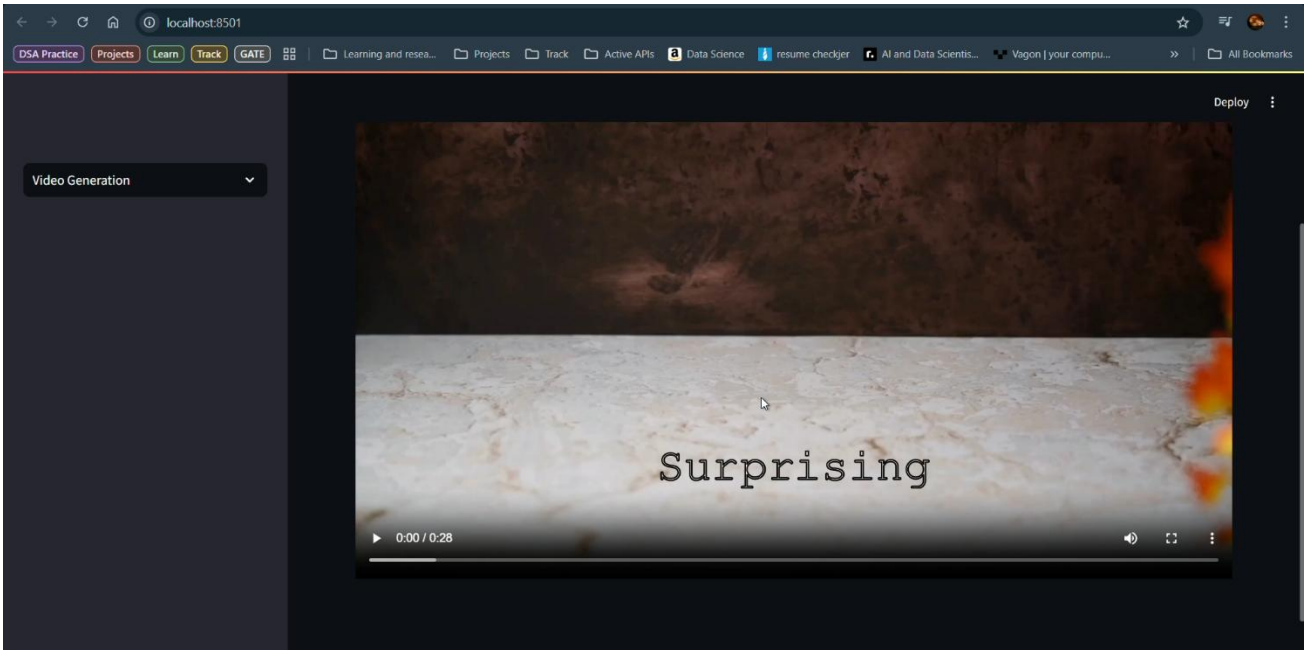


Fig 5.5(f) Generated Video Successfully Displayed to The User

CHAPTER 6
SYSTEM TESTING

6.1 Introduction

System testing plays a crucial role in validating the stability, reliability, and user experience of the AI-Driven Automated Video Generator application. The testing phase ensures that all backend components (such as the script generator, voiceover synthesizer, caption generator, and video renderer) and frontend components (such as the Streamlit-based UI, input fields, buttons, and navigation flow) function correctly and cohesively under various conditions. Each module was evaluated for its ability to handle user inputs, maintain state across stages, and produce accurate, synchronized media outputs. Thorough testing was carried out to assess performance, functionality, and error handling, ensuring a smooth and intuitive experience for end users.

6.2 Test Cases

The following test cases were implemented and successfully passed during testing:•

Input Validation in Script Generation:

Verified that an empty topic input displays a warning message prompting the user to enter valid input.

Tested whitespace-only input and confirmed the same validation behavior.

Script Generation Functionality:

Ensured that clicking "Generate Script" with valid input calls the generate_script() function and displays the output in the text area.

Confirmed that the script is stored in session state for further processing.

Stage Transition – Script to Voiceover:

Verified that clicking "Next" on the Script Generation stage transitions correctly to the Voiceover stage only if a script is present.

Confirmed that the script is displayed on the voiceover page.

Voiceover Generation with Valid Input:

Tested voice selection based on language and gender dropdowns.

Ensured that clicking "Generate Voiceover" triggers audio generation and plays the resulting file.

Audio Playback Validation:

Confirmed that the generated audio_tts.wav plays without errors in the Streamlit audio player.

Stage Transition – Voiceover to Video Generation:

Validated that "Next" moves to the Video Generation stage only if audio was successfully generated.

Ensured state consistency for audio and script values across transitions.

Video Generation Trigger:

Verified that clicking "Next" in Video Generation calls all required functions: generate_timed_captions, getVideoSearchQueriesTimed, generate_video_url, and get_output_media.

Checked that the video is rendered and displayed if all previous stages completed successfully.

Incomplete Workflow Handling:

Confirmed that the app shows a warning if the user attempts to generate a video without completing the script or voiceover stages.

6.3 Results and Discussions

All features were tested in the context of expected user flows.

The session-based navigation system proved reliable, and the app correctly handled valid and invalid inputs.

Voiceover generation using EdgeTTS and audio playback worked seamlessly.

Video rendering triggered correctly using test scripts and confirmed the integration of background video, captions, and audio.

6.4 Performance Evaluation

- **Script Generation Speed:**
- Average time: 2–3 seconds using OpenAI's ChatGPT API.
- **Voiceover Generation Time:**
- Text-to-speech conversion completed within 4–6 seconds for ~1-minute script.
- **Video Rendering Time:**
- Full rendering (audio + captions + video) completed within 1.5–2 minutes.

• **Responsiveness:**

• The Streamlit UI remained responsive during script generation, audio processing, and video rendering stages.

6.5 Summary

System testing confirmed that the AI Video Generator application is robust, reliable, and user-friendly. The application:

• Validates user inputs effectively

• Generates consistent scripts and voiceovers

• Ensures a smooth workflow across stages

• Handles missing data gracefully

• Delivers accurate captions and smooth video compilation

This system is ready for deployment in content creation environments with minimal training.

CHAPTER 7
CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

The **AI-Driven Automated Video Generator for Content Creation** represents a comprehensive solution to one of the most time-consuming and resource-intensive challenges in digital media production — transforming ideas into complete, videos with minimal manual intervention. By integrating multiple AI technologies into a unified platform, this system significantly **reduces the need for complex software tools** and human resources typically involved in **scripting, voiceover, captioning, and video editing.**

The application is designed to operate in three intuitive stages: script generation, voiceover synthesis, and video compilation. **Each stage is driven by advanced AI models and APIs** — including **ChatGPT** for **natural language generation, EdgeTTS** for lifelike speech synthesis, and Whisper for accurate, timed captioning. The final video assembly, powered by **MoviePy** and **FFmpeg**, intelligently combines the script, narration, and visuals retrieved from the **Pexels API** to produce coherent, visually appealing video content that can be used across **educational, marketing, and social platforms.**

One of the standout features of this system is its **simplicity and accessibility**. Developed using **Streamlit**, the interface is user-friendly and tailored to both technical and non-technical users, making it possible for **individuals with no prior video editing experience to generate high-quality content with just a few inputs**. Additionally, the **modular architecture** ensures that each component functions independently, allowing for **future enhancements** such **as multilingual support, alternative AI models** and integration with additional media libraries.

The project's architecture reflects best practices in **AI integration** and software design. With the use of session state handling, dynamic input management, and asynchronous audio processing, the platform delivers a responsive and efficient experience. Resource optimization has been taken into consideration, making it viable to run on systems with moderate hardware configurations while still ensuring real-time feedback and media generation.

In a world where content is increasingly consumed in multimedia formats, this system offers a scalable, intelligent alternative to traditional content creation workflows. It empowers educators to generate informative lessons, marketers to produce branded videos, and content creators to transform ideas into shareable stories — all in a matter of minutes. Beyond its current functionality, the platform serves as a strong foundation for future research and development in generative multimedia, AI-powered storytelling, and automated creative tools.

In summary, the AI-Driven Automated Video Generator exemplifies the fusion of artificial intelligence and media technology in a practical, impactful, and extensible application. It not only enhances productivity and creativity but also paves the way for the next generation of intelligent content creation systems.

CHAPTER 8
REFERENCES

- **AI-Powered Content Creation** – A study on **Generative AI in multimedia production**.
- **Text-to-Video AI Models** – Research on **Natural Language Processing (NLP) and Deep Learning** for automated video synthesis.
- **Ethical AI Guidelines for Content Creation** – IEEE standards ensuring AI-generated media complies with ethical AI guidelines.

LIST OF FIGURES

LIST OF ABBREVIATIONS

S. N	ABBREVIATION	FULL FORM
1	LLM	Large Language Model
2	API	Application Programming Interface
3	JSON	JavaScript Object Notation
4	CLI	Command Line Interface
5	NLP	Natural Language Processing
6	TTS	Text-to-Speech
7	ASR	Automatic Speech Recognition
8	UI	User Interface
9	GPT	Generative Pre-trained Transformer
10	SRS	Software Requirements Specification
11	DFD	Data Flow Diagram
12	FFmpeg	Fast Forward MPEG
13	SSD	Solid State Drive
14	RAM	Random Access Memory